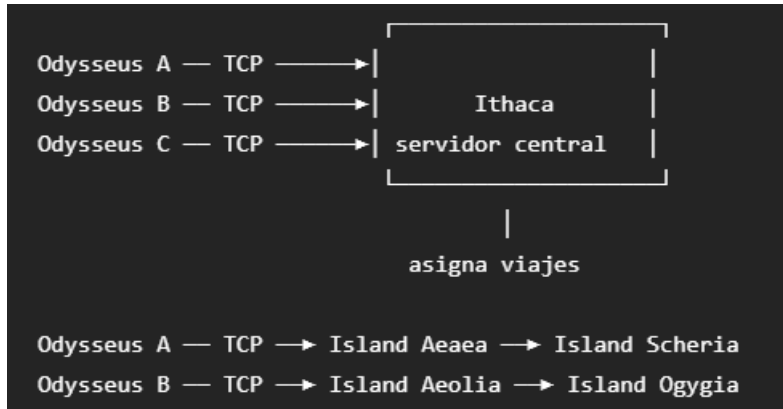


Plantejament de la Practica SO

Plantejament general de com es connectaran les coses:



Ithaca serà el servidor central

Island serà el servidor que representa la illa amb el mercat i el port

Odysseus serà el client que es va manejant pel usuari

1. Ithaca

Ithaca serà un servidor TCP.

```
leer ithaca.dat y voyages.dat
↓
socket()
↓
bind(IP y puerto de Ithaca)
↓
listen()
↓
accept() en bucle
↓
crear un thread para cada Odysseus conectado
```

Ithaca tindrà un socket principal que únicament accepta connexions

Proposta es crear un thread por client

Tots els threads compartiran el mateix array de viatges, per tan necessitem un mutex

2. Island

Cada procés Island serà un servidor TCP

```
leer island.dat y stock.db
validar rutas con SPHRAGIS
↓
socket → bind → listen
↓
accept() en bucle
↓
thread por Odysseus
```

Cada thread de client gestiona:

- petició de arribada
- espera per atracar
- list market
- buy
- sell
- buy map
- deliver
- sortida
- desconnexió

La capacitat del port es gestiona amb el numero de plaças ocupades, llista de barcos atracats i una fifo pels barcos que estan esperant

Tambe hem de utilitzar mutex per el numero de plaças ocupades, la cua de la FIFO i la entrada i sortida dels barcos. També un mutex pel stock

3. Odysseus

Es un client interactiu

```
leer odysseus.dat
↓
mostrar terminal
↓
leer comando
↓
validarlo según el estado
↓
enviar trama por el socket activo
↓
esperar respuesta
```

Mentres espera al port consumeix 7kg per cada 5 segons això ho solucionem amb un select

```
select(socket, timeout de 5 segundos)
|
|— llega una trama → procesar WAIT/DOCKED/error
|
|— expiran 5 s → consumir 7 kg
```

4. Protocols de trama

TYPE	1 byte
FLAGS	1 byte
DATA_LENGTH	1 byte
DATA	253 bytes

S'ha de serialitzar un buffer

Y exige:

- un único `write()` de 256 bytes;
- un único `read()` de 256 bytes;
- padding restante a `0x00` ;
- validar estrictamente `TYPE` , `FLAGS` , longitud y contenido;
- responder con `NACK 0x7F` si la trama no corresponde a ningún formato válido.

La integritat es valida amb md5sum

5. Transferència de fitxers

```
cabecera 0x20
  ↓
READY 0x22
  ↓
N tramas de datos 0x21
  ↓
ACK final 0x23
```

6. Md5sum

```
pipe()
  ↓
fork()
  ↓
hijo: dup2(pipe_write, STDOUT_FILENO)
  ↓
hijo: execlp("md5sum", ...)
  ↓
padre: read(pipe_read, ...)
  ↓
padre: waitpid()
```

7. Senyals

Els processos han de acabar amb ctrl+c, SIGINT

8. Persistència

Ha de persistir el stock de cada island, diners i inventari de cada odysseus i estat necessari per continuar després de reiniciar.

```
configuración inicial    → .dat, solo lectura
stock de Island          → stock.db, lectura y escritura
estado persistente barco → fichero propio de Odysseus
viajes Ithaca            → estructura gestionada por Ithaca
```